UFU - Sistemas Distribuídos

VoteNow - Sistema de Votação Distribuído

Arquitetura, implementação, fundamentos teóricos e validação final

Monte Carmelo - 26/02/2026

Resumo

Este relatório técnico descreve um projeto de sistema de votação distribuído (VoteNow) e compara etapas anteriores da evolução arquitetônica. As etapas anteriores abordam uma construção clássica com sockets TCP, NameServer, eleição de líder, heartbeat, relógios lógicos e exclusão mútua distribuída. A etapa atual migra para uma arquitetura orientada a mensagens, apoiada em middleware NATS/JetStream com streams, consumidores duráveis e buckets Key-Value para estado operacional e de votação.

O documento apresenta: objetivos do projeto, implementação dos conceitos de sistemas distribuídos na etapa anterior, integração de middleware na fase atual, resultados de validação, limitações técnicas e instruções completas de construção e execução.

Sumário

1 Tema, Objetivos e Visão Geral do Projeto **3**
1.1 Tema e problema . . . 3
1.2 Objetivos funcionais . . . 3
1.3 Objetivos não funcionais . . . 3
1.4 Justificativa para sistemas distribuídos . . . 3
1.5 Arquitetura geral da solução . . . 3
1.6 Workflow estruturado do Voto . . . 4

2 Implementação dos Conceitos de Sistemas Distribuídos **5**
2.1 Processos e Threads . . . 5
2.2 Comunicação via Sockets . . . 6
2.3 Arquitetura Baseada em Mensagens . . . 6
2.4 Nomenclatura de Processos . . . 6
2.5 Sincronização . . . 6
2.5.1 Relógios lógicos . . . 6
2.5.2 Exclusão mútua . . . 7

3 Seção de Middleware **7**
3.1 Definição e relevância . . . 7
3.2 Integração ao projeto . . . 7
3.3 Problemas resolvidos e melhorias . . . 8
3.4 Consistência, replicação, tolerância a falhas e segurança . . . 8
3.5 Desafios e aprendizados . . . 8
3.6 Validação da sincronização e exclusão mútua . . . 9

4 Instruções Detalhadas de Construção e Execução (Obrigatório) **9**
4.1 Pré-requisitos . . . 9
4.2 Visão Estrutural da Infraestrutura (Docker Compose) . . . 9
4.3 Construção e execução . . . 10

5 Conclusão e Encerramento **11**
5.1 Síntese do que foi entregue . . . 11
5.2 Validação final (o que foi observado) . . . 11

1 Tema, Objetivos e Visão Geral do Projeto

1.1 Tema e problema

O tema do projeto é a construção de um **sistema de votação distribuído** com cliente iOS, painel web e backend com múltiplos nós. O problema central é garantir apuração consistente e operação contínua, mesmo com concorrência e falhas parciais.

1.2 Objetivos funcionais

- registrar votos com identificação única;
- consultar resultados agregados;
- suportar monitoramento e operações de controle (reset, parada e recuperação de nós);
- manter o funcionamento sob a indisponibilidade parcial de componentes.

1.3 Objetivos não funcionais

- tolerância a falhas e recuperação operacional;
- escalabilidade horizontal de serviços;
- rastreabilidade por contratos de mensagem;
- redução de acoplamento entre cliente e servidores de apuração.

1.4 Justificativa para sistemas distribuídos

A natureza do problema exige múltiplos nós cooperando: há concorrência de votos, necessidade de alta disponibilidade e separação entre plano de dados (votos) e plano de controle (operação dos nós). Sistemas distribuídos permitem tratar esses requisitos com mecanismos de coordenação e replicação.

1.5 Arquitetura geral da solução

A evolução do projeto é marcada pela transição de uma arquitetura baseada em coordenação direta para uma arquitetura baseada em eventos e mensageria:

Etapas anteriores (Coordenação Direta):

- NameServer centralizado para a descoberta dinâmica do líder;

- Três servidores de apuração (*tally servers*: s1, s2, s3) implementando algoritmos de eleição, heartbeat, exclusão mútua distribuída e replicação de dados em memória via conexões TCP;
- Comunicação síncrona: interface web operando via HTTP e o aplicativo iOS enviando requisições via TCP puro.

Etapa atual (Arquitetura Orientada a Mensagens - Etapa 5): A arquitetura foi refatorada e dividida em três camadas lógicas principais para promover o desacoplamento e a persistência resiliente:

Camada de Clientes

Composta pelo aplicativo iOS, que agora se comunica através de NATS/TCP (requisição/resposta e publicação/assinatura), e pelo Painel Web, integrado em tempo real via WebSockets (`nats.ws`).

Camada de Mensageria e Estado

Sustentada por um cluster NATS/JetStream (3 nós) que atua como barramento central. Ele é responsável por manter o Stream de persistência `VOTE_CMD` (com consumidores duráveis) e os Buckets de Key-Value (KV) que armazenam o estado da votação de forma distribuída.

Camada de Serviços (Microsserviços)

Funções antes aglutinadas foram divididas em serviços especializados:

- `vote-api`: responsável por receber comandos e consultas, validando-os antes de publicá-los no barramento;
- `vote-processor`: consome as mensagens de forma durável, garantindo processamento com idempotência;
- `vote-admin` e `monitores`: isolam o gerenciamento de *locks* administrativos, *reset* do sistema e controle de concorrência por *lease* (liderança operacional).

1.6 Workflow estruturado do Voto

Para garantir rastreabilidade e consistência desde o cliente até a consolidação dos dados, o processamento de um voto segue um fluxo unidirecional rigoroso:

1. **Submissão:** O cliente (aplicativo iOS ou interface Web) gera e transmite o evento de voto contendo a identificação única.
2. **Validação e Roteamento:** O serviço `vote-api` intercepta a requisição, realiza a validação do *payload* e publica a intenção de voto no barramento NATS.

3. **Persistência Durável:** O middleware JetStream recebe a mensagem e a persiste imediatamente no Stream `VOTE_CMD`, garantindo que a requisição não seja perdida em caso de falha de serviço.

4. **Consumo e Processamento:** De forma assíncrona e segura, o serviço `vote-processor` atua como um consumidor durável, retirando a mensagem do Stream e processando-a sob critérios de idempotência.

5. **Atualização de Estado (CAS):** O processador atualiza os contadores e o status da votação registrados nos buckets de armazenamento Key-Value (ex: `VOTE_COUNT` e `VOTE_STATE`). Esta operação utiliza *Compare-And-Swap* (CAS) e verificação de revisão para impedir condições de corrida simultâneas.

6. **Propagação do Resultado:** Uma vez confirmado o novo estado no KV, eventos de atualização são disparados, permitindo que o Painel Web e o Cliente iOS recebam e exibam imediatamente o resultado agregado.

2 Implementação dos Conceitos de Sistemas Distribuídos

2.1 Processos e Threads

As etapas anteriores organizam os processos em `nameserver`, `s1`, `s2`, `s3` e `web`. Nos tally servers, threads são usadas para:

- escuta de conexões TCP;
- servidor HTTP local;
- envio e monitoramento de heartbeat;
- timers de eleição e coordenação.

Controle de concorrência:

- exclusão mútua distribuída por Ricart-Agrawala;
- sincronização local com locks em estruturas compartilhadas.

Vantagens: algoritmos clássicos.
Limitações: maior complexidade operacional e acoplamento entre responsabilidades.

2.2 Comunicação via Sockets

O protocolo utilizado é **TCP**. Os nós usam sockets para troca de mensagens JSON com envelope contendo tipo, nome, remetente, payload e timestamps lógicos.

Tratamento de erros e desconexões:

- timeouts de envio/recebimento;
- eleição disparada por timeout de heartbeat;
- retries no cliente iOS ao detectar troca de líder.

2.3 Arquitetura Baseada em Mensagens

O modelo é **híbrido**: cliente-servidor com coordenação entre pares no backend.

Estrutura das mensagens:

- tipos: `COMMAND`, `EVENT`, `QUERY`, `RESPONSE`;
- mensagens distribuídas: `ELECTION`, `OK`, `COORDINATOR`, `REQUEST`, `REPLY`, `HEARTBEAT`.

Serialização e desserialização: JSON.

Garantias: entrega e ordenação dependem de TCP e da lógica da aplicação; não há broker com persistência nativa.

2.4 Nomenclatura de Processos

- nós de apuração identificados como `s1`, `s2`, `s3`;
- registro de serviço em formato `tally-{server}-{room}`;
- descoberta de líder e serviços via NameServer.

2.5 Sincronização

2.5.1 Relógios lógicos

As etapas anteriores utilizam:

- **Relógio de Lamport** para ordenação lógica total;
- **Relógio vetorial** para causalidade e detecção de eventos concorrentes.

Conflitos de causalidade são detectados pelo `StateManager` quando vetores são concorrentes.

2.5.2 Exclusão mútua

O algoritmo adotado é **Ricart-Agrawala**. O acesso à seção crítica é controlado por mensagens `REQUEST/REPLY`, com prioridade por timestamp de Lamport e desempate por identificador do processo.

3 Seção de Middleware

3.1 Definição e relevância

O middleware adotado na etapa atual é o **NATS + JetStream**. Ele é relevante por fornecer:

- barramento unificado para request/reply e publish/subscribe;
- persistência de mensagens e estado distribuído;
- mecanismos de coordenação sem reimplementar toda a lógica distribuída na camada de domínio.

3.2 Integração ao projeto

A integração foi realizada em três níveis.

Infraestrutura:

- cluster NATS com 3 instâncias;
- autenticação por usuários e ACL por subject;
- WebSocket para o painel web.

Provisionamento:

- stream `VOTE_CMD` e consumer durável `VOTE_CMD_PROC`;
- buckets KV: `KV_VOTE_STATE`, `KV_VOTE_COUNT`, `KV_CONTROL`, `KV_MONITOR_STATE`, `KV_MONITOR_LEADER`, `KV_MONITOR_CONTROL`.

Serviços:

- `vote-api`: entrada de comandos e consultas;
- `vote-processor`: processamento idempotente;
- `vote-admin`: lock administrativo e controle operacional;
- `monitor-node`: heartbeat e liderança por lease.

3.3 Problemas resolvidos e melhorias

A etapa atual supera limitações encontradas nas etapas anteriores ao:

- reduzir acoplamento ponto a ponto;
- centralizar contratos de mensagem;
- persistir estado e contadores em camada replicada;
- separar responsabilidades de API, processamento, administração e monitoramento.

3.4 Consistência, replicação, tolerância a falhas e segurança

Consistência

Atualizações em KV usam controle por revisão (CAS), evitando sobrescrita concorrente silenciosa. O processamento de voto mantém fluxo de estado idempotente.

Replicação

Streams e buckets são provisionados com réplicas (configuração padrão igual a 3), distribuindo dados no cluster.

Tolerância a falhas

- reconexão automática em clientes NATS;
- liderança operacional por lease com expiração;
- fallback para servidor saudável quando liderança registrada não é válida.

Segurança

- autenticação por usuário/senha e permissões por subject;
- separação de perfis (web, iOS, backend, ops);
- observação: WebSocket foi configurado com `no_tls: true`, garantindo execução em ambiente localhost para apresentação em sala.

3.5 Desafios e aprendizados

Desafios:

- migração de lógica de coordenação para semântica de middleware;
- tratamento de corridas de inicialização do JetStream;

- padronização de schemas e subjects.

Aprendizados:

- middleware simplifica domínio, mas exige governança de contrato;
- idempotência e observabilidade são requisitos de primeira ordem em sistemas de voto;
- operação distribuída deve ser tratada como parte da arquitetura, não como pós-processo.

3.6 Validação da sincronização e exclusão mútua

A validação de sincronização é suportada por dois mecanismos explícitos:

- Etapas anteriores: ordenação lógica e mutex distribuído (Lamport, vetorial e Ricart-Agrawala);
- Etapa atual: CAS em KV, lock com fencing token e lease de liderança.

4 Instruções Detalhadas de Construção e Execução (Obrigatório)

4.1 Pré-requisitos

- Docker e Docker Compose;
- Python 3.11+;
- Node.js 20+ e npm;
- Xcode para execução do cliente iOS.

4.2 Visão Estrutural da Infraestrutura (Docker Compose)

Para fins de simplificação no ambiente de desenvolvimento e simulação, toda a arquitetura foi orquestrada através do Docker Compose. O mapa lógico das dependências entre os contêineres é estabelecido da seguinte forma:

Cluster Base de NATS e JetStream

Três contêineres principais (`nats-1`, `nats-2` e `nats-3`) inicializam o cluster do barramento. Eles se comunicam entre si internamente para sincronização de dados e expõem portas padronizadas para acesso dos microsserviços e clientes:

- **4222:** Conexões de clientes e serviços (padrão NATS).
- **6222:** Roteamento interno do cluster.
- **8222:** Monitoramento e HTTP de gerenciamento.

Serviços Backend (Contêineres Internos)

Os microsserviços foram conteinerizados separadamente e conectam-se ativamente ao cluster do NATS (direcionados principalmente através de portas internas na rede Docker). Estão divididos em:

- `vote-api` e `vote-processor`: núcleo de validação e ingestão de estado.
- `vote-admin`: painel operacional administrativo oculto para operações críticas.
- `monitor-s1`, `monitor-s2`, `monitor-s3`: instâncias de monitoramento responsáveis pelo processo de *lease* do líder ativo no cluster.

Serviço Frontend e Clientes

- `web`: O servidor web é exposto na porta **3000**. Ele é acessado através de um navegador comum e conecta-se nativamente ao NATS utilizando WebSockets.
- **Cliente iOS**: Não compõe a *stack* Docker. Opera de forma autônoma no simulador ou dispositivo físico e se conecta via TCP puro à porta 4222 exposta pelo cluster NATS.

4.3 Construção e execução

1) Subir toda a stack

```
docker compose up --build -d
```

2) Verificar estado dos serviços

```
docker compose ps
docker compose logs -f nats-1 nats-2 nats-3 vote-api vote-processor vote-admin
   monitor-s1 monitor-s2 monitor-s3 web
```

3) Acessar aplicações

- painel web: http://localhost:3000

4) Executar validações básicas

```
python3 -m compileall -q server
cd web && npm run build
```

5) Simular falhas operacionais

No painel web:

- usar ação de derrubar servidor monitor;
- confirmar mudança de status e líder;
- usar ação de reativar servidor.

5 Conclusão e Encerramento

A evolução das etapas anteriores para a Etapa 5 representa uma transição de uma coordenação *artesanal* (ponto a ponto via TCP, com eleição, heartbeat e exclusão mútua distribuída) para uma arquitetura *orientada a mensagens* com **middleware especializado** (NATS/JetStream), preservando fundamentos de sistemas distribuídos e reduzindo acoplamento operacional.

5.1 Síntese do que foi entregue

- **Desacoplamento e governança de contrato:** subjects, streams e consumidores duráveis padronizam a troca de mensagens e facilitam evolução do sistema.
- **Persistência e recuperação:** comandos de voto permanecem no JetStream, permitindo retomada controlada após falhas.
- **Consistência operacional:** estado e contadores em KV com revisão (CAS) mitigam sobrescritas concorrentes e suportam idempotência no processamento.
- **Operação e observabilidade:** separação de responsabilidades (`vote-api`, `vote-processor`, `vote-admin`, `monitor-*`) torna diagnóstico e controle mais previsíveis.

5.2 Validação final (o que foi observado)

Durante testes de execução e simulação de falhas, o sistema manteve o fluxo de votação operante com **reconexão automática** e **continuidade do estado** em cenários de reinício de serviços. A liderança baseada em *lease* evitou responsabilidades conflitantes simultâneas, e o uso de CAS em KV reduziu condições de corrida na apuração.

Encerramento

Didaticamente e na prática, o projeto VoteNow evidencia duas formas de materializar os mesmos conceitos de sistemas distribuídos: (1) algoritmos clássicos implementados na

aplicação e (2) mecanismos providos por middleware moderno. O resultado final é uma solução mais modular, mais simples de operar e com caminho claro para evolução segura.